PHENIKAA UNIVERSITY
School of Computing

CSE702060 – TRỰC QUAN HOÁ DỮ LIỆU

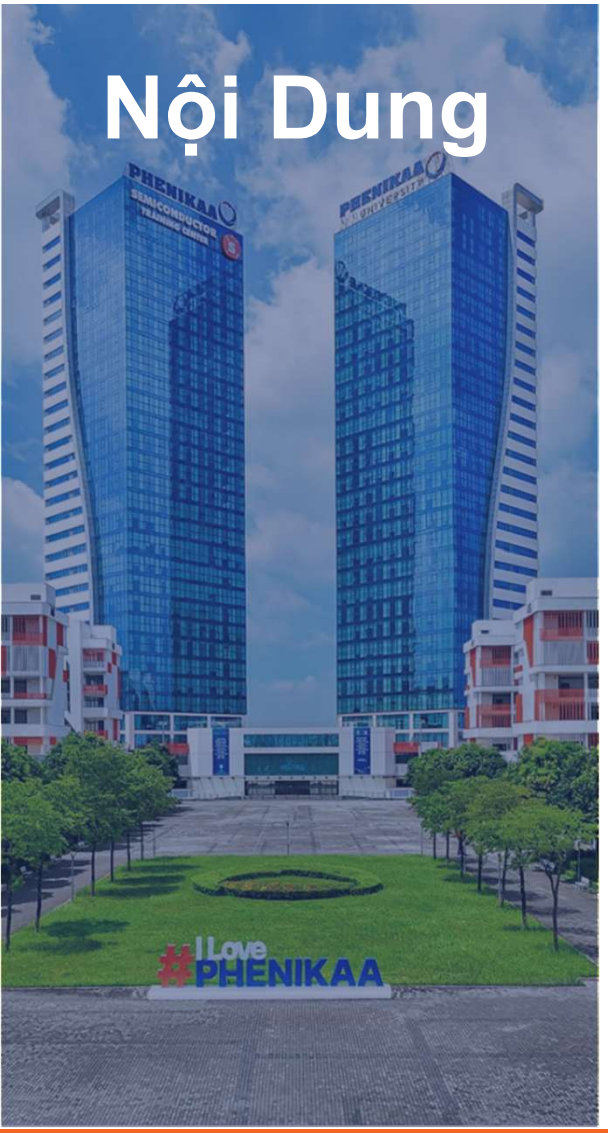
TRỰC QUAN HÓA DỮ LIỆU PHÂN CẤP, MẠNG, LUỒNG

Giảng viên: Nguyễn Ngọc Giang
Contact: giang.nguyenngoc@phenikaa-uni.edu.vn

Mục tiêu

PHENIKAA UNIVERSITY
School of Computing

- **Nhận diện** dữ liệu phân cấp, dữ liệu mạng và dữ liệu luồng.
- **Phân biệt** các thành phần cơ bản của từng loại dữ liệu.
- **Lựa chọn** dạng trực quan phù hợp với cấu trúc dữ liệu và câu hỏi phân tích.
- **So sánh** ưu điểm, nhược điểm của tree diagram, dendrogram, treemap, sunburst, node-link diagram, adjacency matrix, Sankey và alluvial diagram.
- **Phát hiện** các vấn đề về khả năng đọc khi dữ liệu có nhiều nút, cạnh hoặc luồng.
- **Xây dựng** một sản phẩm trực quan đơn giản và giải thích được các quyết định thiết kế.



Nội Dung

PHENIKAA UNIVERSITY
School of Computing

1. Trực quan dữ liệu phân cấp
2. Trực quan dữ liệu mạng
3. Trực quan dữ liệu luồng
4. Một số lưu ý chọn và vẽ biểu đồ
5. Thực hành

1. Trực quan dữ liệu phân cấp

Khái niệm dữ liệu phân cấp

PHENIKAA UNIVERSITY
School of Computing

- Dữ liệu phân cấp là dữ liệu trong đó các đối tượng được tổ chức thành nhiều cấp. Mỗi phần tử, ngoại trừ nút gốc, thường có một nút cha.
- Ví dụ: Trường đại học có tổ chức như sau:

Khoa	Ngành	Chuyên ngành	Số sinh viên
Công nghệ	CNTT	Khoa học dữ liệu	80
Công nghệ	CNTT	Kỹ thuật phần mềm	120
Công nghệ	Điện tử	Hệ thống nhúng	60
Kinh tế	Quản trị	Marketing	110
Kinh tế	Quản trị	Nhân sự	70
Kinh tế	Tài chính	Ngân hàng	90

Thành phần cơ bản

PHENIKAA UNIVERSITY
School of Computing

1. Nút gốc

- Là nút cao nhất trong cấu trúc.
- Không có nút cha.
- Đại diện cho toàn bộ hệ thống.
- Ví dụ: “Trường đại học” là nút gốc trong sơ đồ tổ chức.

2. Nút cha và nút con

- Nút cha chứa hoặc quản lý một hay nhiều nút con.
- Nút con có thể tiếp tục trở thành nút cha ở cấp thấp hơn.

Thành phần cơ bản

PHENIKAA UNIVERSITY
School of Computing

3. Nút lá

- Là nút không có nút con.
- Thường đại diện cho đơn vị chi tiết nhất.

4. Cấp độ phân cấp

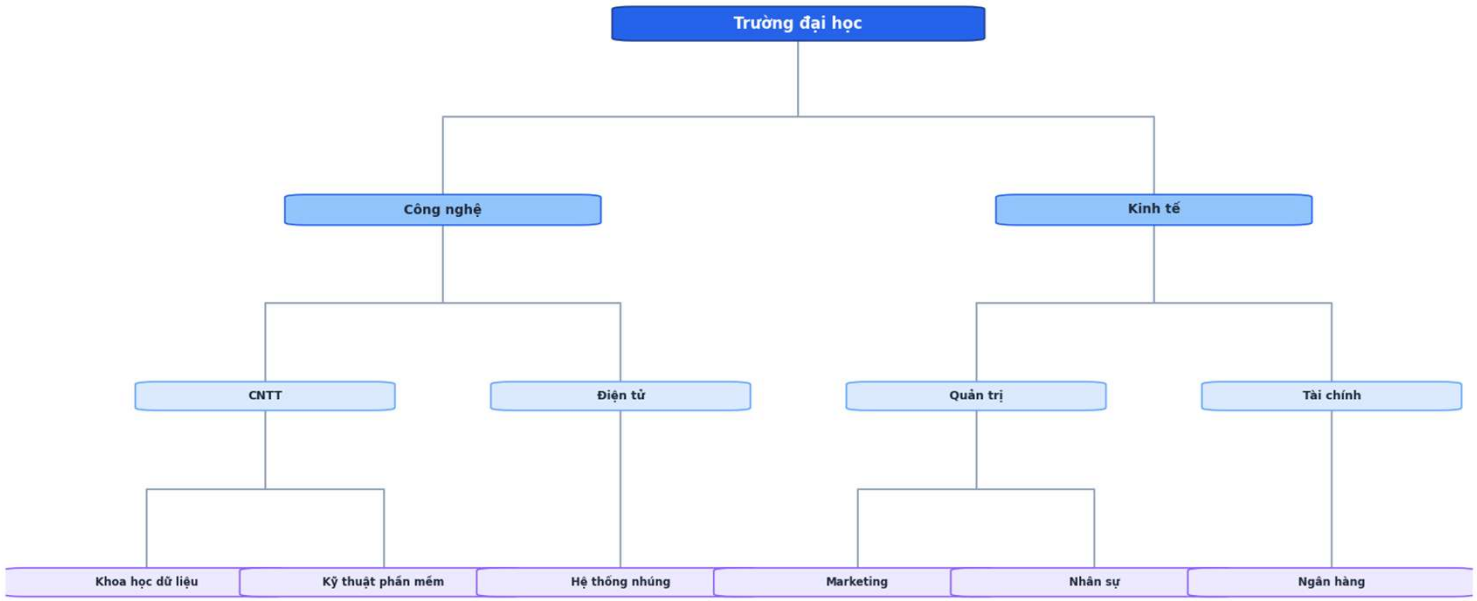
- Khoảng cách từ nút gốc đến một nút.
- Cấp độ giúp người xem hiểu vị trí của đối tượng trong hệ thống.

Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

Tree diagram

Tree diagram thể hiện cấu trúc phân cấp bằng các nút và đường nối.



Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

Tree diagram

- **Phù hợp khi**
  - Cần nhìn rõ quan hệ cha, con.
  - Cấu trúc có số lượng nút vừa phải.
  - Cần theo dõi đường đi từ nút gốc đến nút lá.
  - Giá trị định lượng không phải là trọng tâm chính.
- **Ưu điểm**
  - Trực quan, dễ hiểu.
  - Thể hiện rõ các cấp.
  - Thuận lợi khi giải thích cấu trúc tổ chức.
- **Hạn chế**
  - Chiếm nhiều không gian.
  - Khó đọc khi cây quá sâu hoặc quá rộng.
  - Các nhánh có thể chồng lấn.
  - Khó so sánh giá trị định lượng giữa các nút.
- **Ví dụ sử dụng**
  - Sơ đồ tổ chức.
  - Cấu trúc thư mục.
  - Phân loại sinh vật.
  - Danh mục sản phẩm.

Cách vẽ

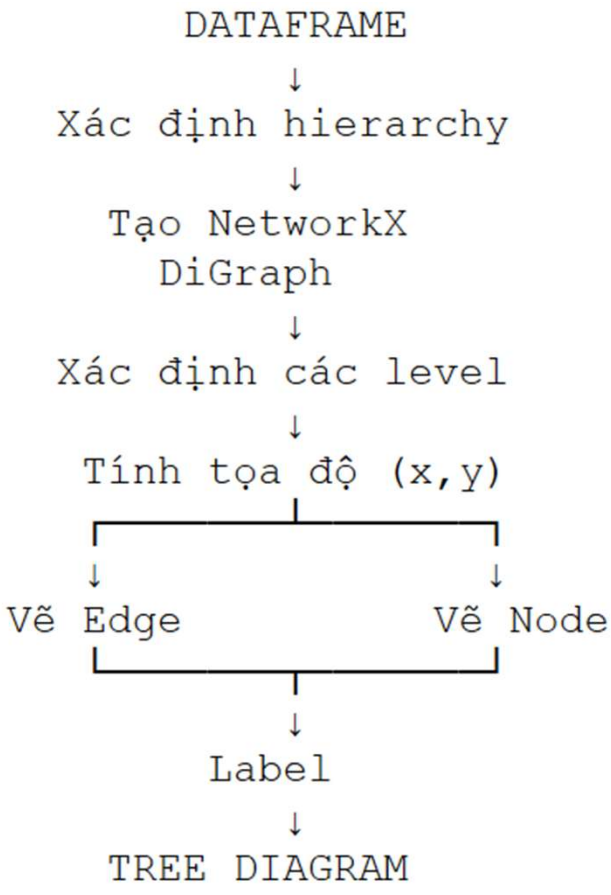
PHENIKAA UNIVERSITY
School of Computing

- Sử dụng thư viện NetworkX, hàm NX.DiGraph() tạo graph

```
import networkx as nx

G = nx.DiGraph()

G.add_edge("Trường đại học", "Công nghệ")
G.add_edge("Trường đại học", "Kinh tế")
G.add_edge("Công nghệ", "CNTT")
G.add_edge("Công nghệ", "Điện tử")
G.add_edge("Kinh tế", "Quản trị")
G.add_edge("Kinh tế", "Tài chính")
G.add_edge("CNTT", "Khoa học dữ liệu")
G.add_edge("CNTT", "Kỹ thuật phần mềm")
G.add_edge("Điện tử", "Hệ thống nhúng")
G.add_edge("Quản trị", "Marketing")
G.add_edge("Quản trị", "Nhân sự")
G.add_edge("Tài chính", "Ngân hàng")
```



Cách vẽ

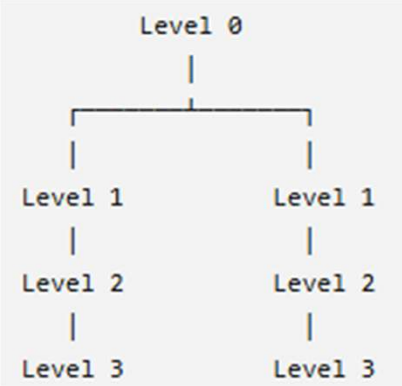
- Xác định level

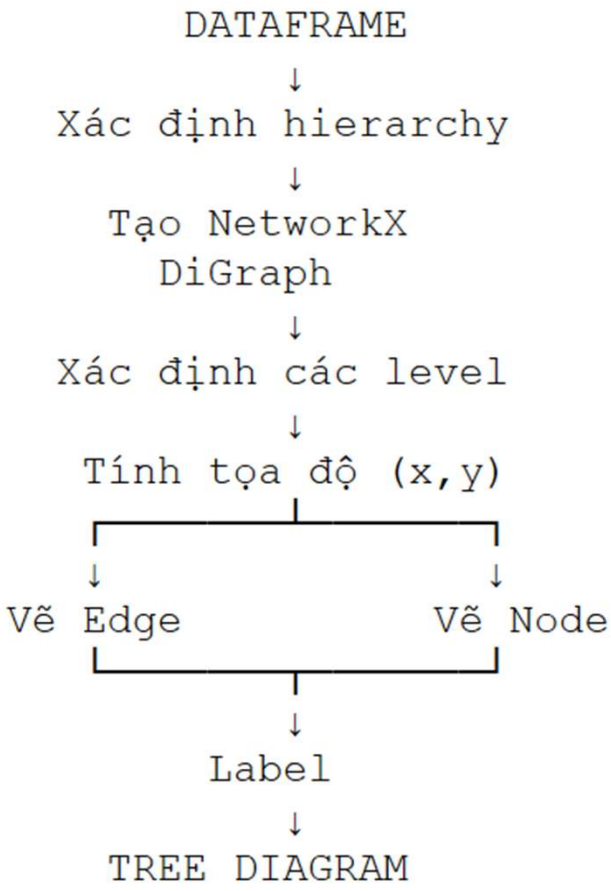
```
levels = {
    "Trường đại học": 0,

    "Công nghệ": 1,
    "Kinh tế": 1,

    "CNTT": 2,
    "Điện tử": 2,
    "Quản trị": 2,
    "Tài chính": 2,

    "Khoa học dữ liệu": 3,
    "Kỹ thuật phần mềm": 3,
    "Hệ thống nhúng": 3,
    "Marketing": 3,
    "Nhân sự": 3,
    "Ngân hàng": 3
}
```



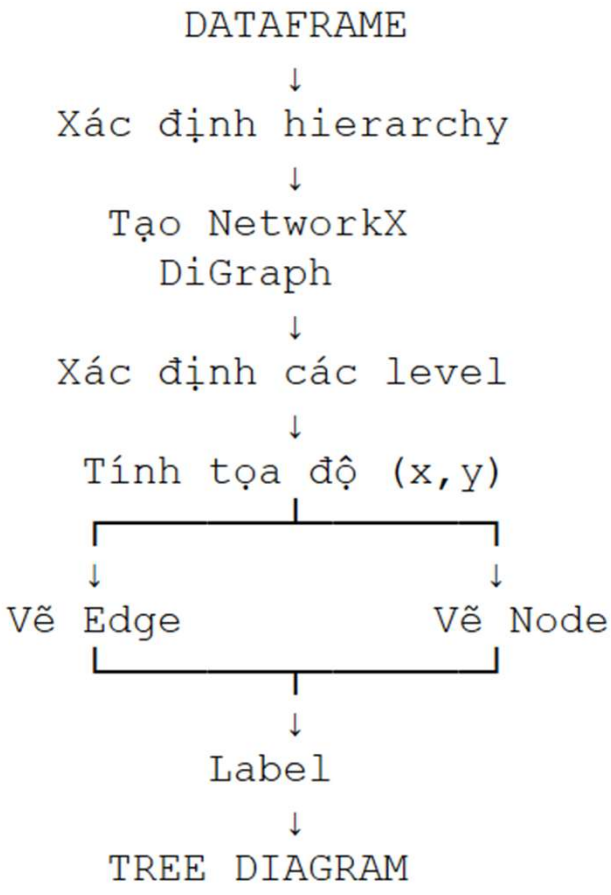


Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- Xác định vị trí các node (tọa độ x, y)
  - Cùng level → cùng y
  - Các node cùng level → phân bố theo x
  - Node cha → đặt ở giữa các node con

```
def calculate_positions(G, root):
    pos = {}
    #Viết code tính tọa độ các node
    #pos[node] = (x, -level)
    ...
    return pos
```



Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- Vẽ node
  - Node tròn

```
nx.draw_networkx_nodes()
```

  - Node chữ nhật

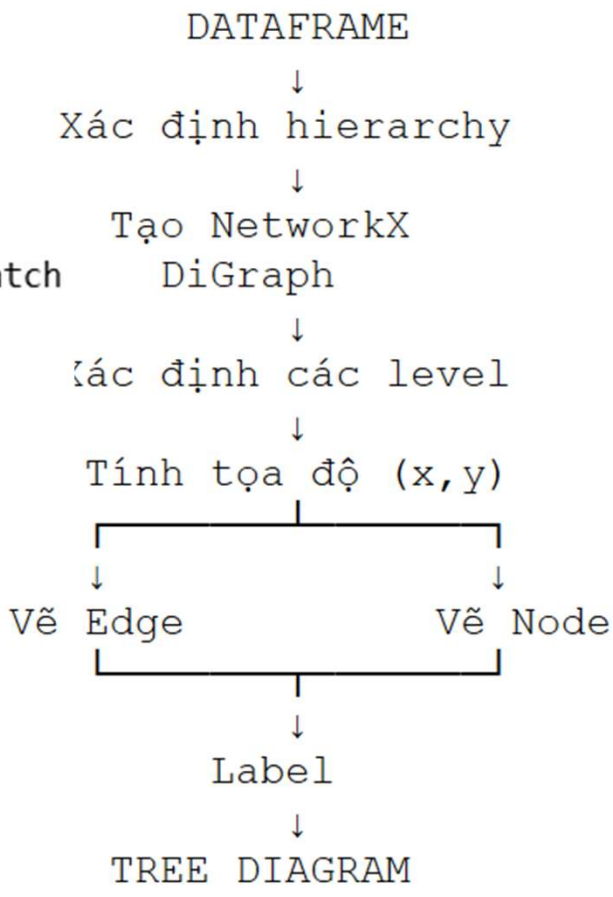
```
from matplotlib.patches import FancyBboxPatch

box = FancyBboxPatch(...)
ax.add_patch(box)
ax.text(...)
```

- Vẽ cạnh

```
x1, y1 = pos[parent]
x2, y2 = pos[child]
```

```
ax.plot(
    [x1, x1, x2, x2],
    [y1, mid_y, mid_y, y2]
)
```

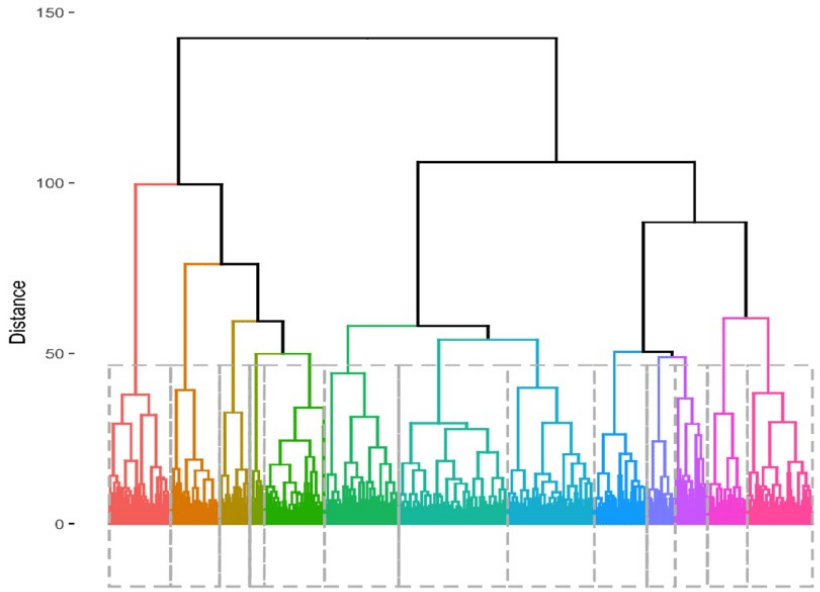


Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

Dendrogram

Dendrogram là sơ đồ cây thường được sử dụng để thể hiện quá trình phân nhóm hoặc mức độ tương đồng.





Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

- **Đặc điểm**
  - Các đối tượng được đặt ở đầu hoặc cuối các nhánh.
  - Khoảng cách nối giữa các nhánh có thể biểu diễn mức độ khác biệt.
  - Thường xuất hiện trong phân cụm phân cấp.
- **Phù hợp khi**
  - Cần trình bày kết quả hierarchical clustering.
  - Cần quan sát nhóm nào được kết hợp trước.
  - Cần lựa chọn số cụm dựa trên khoảng cách.
- **Câu hỏi phân tích**
  - Hai đối tượng nào giống nhau nhất?
  - Nhóm nào được hình thành sớm nhất?
  - Có thể chia dữ liệu thành bao nhiêu cụm?
  - Ngưỡng cắt dendrogram ảnh hưởng thế nào đến số cụm?
- **Hạn chế**
  - Khó đọc khi có nhiều đối tượng.
  - Người xem cần hiểu ý nghĩa của chiều cao hoặc khoảng cách nối.
  - Thứ tự trình bày các nút lá có thể làm người xem cảm nhận sai về khoảng cách.

Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- Sử dụng thư viện Scipy và matplotlib

```
from scipy.cluster.hierarchy import dendrogram, linkage
```

- 2 hàm quan trọng
  - Chuyển dữ liệu thành Hierarchical Cluster: *linkage()*
  - Chuyển Hierarchical Clusters thành Dendrogram: *dendrogram()*

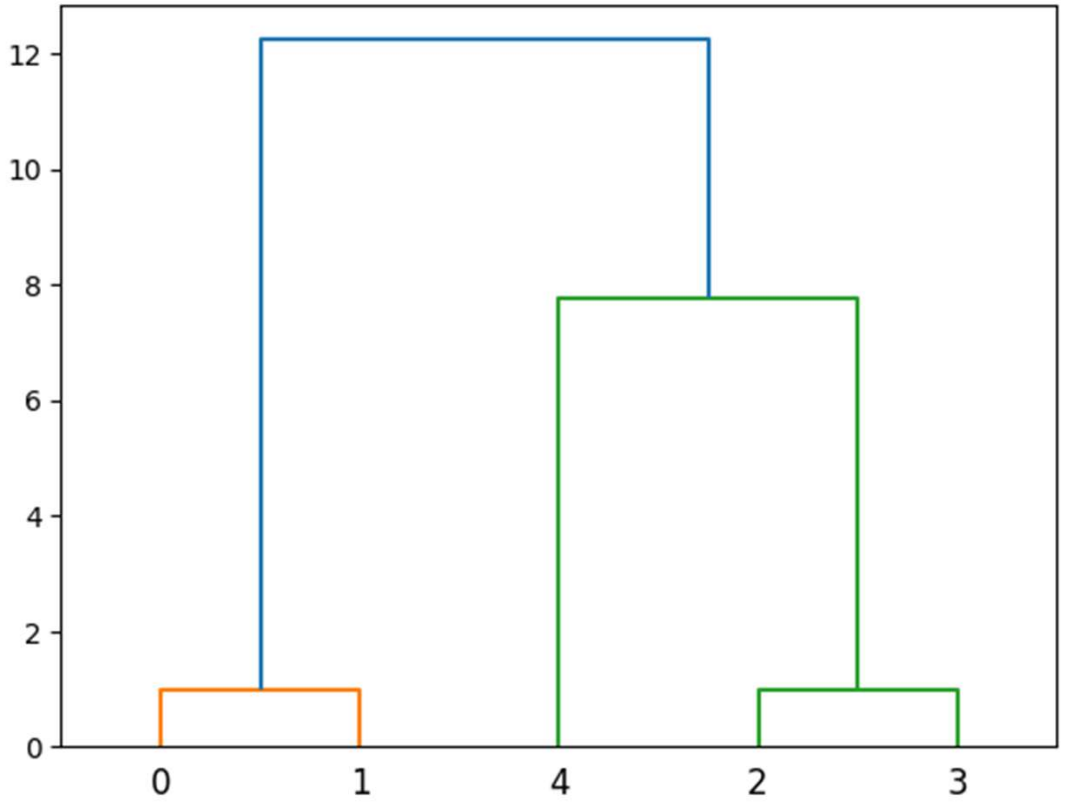
Cách vẽ

```
X = np.array([
    [1, 1],
    [2, 1],
    [5, 5],
    [6, 5],
    [10, 10]
])

Z = linkage(X, method="ward")

dendrogram(Z)

plt.show()
```



Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- linkage() tạo ra một linkage matrix Z

```
[[0.          1.          1.          2.        ]
 [2.          3.          1.          2.        ]
 [4.          5.         10.          3.        ]
 [6.          7.         15.          5.        ]]
```

với

```
Z[i] = [cluster_1, cluster_2, distance, number_of_observations]
```

Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- linkage(method=…)

Method	Ý tưởng
single	khoảng cách nhỏ nhất giữa hai cluster
complete	khoảng cách lớn nhất
average	khoảng cách trung bình
ward	tối thiểu hóa variance khi gộp cluster
centroid	khoảng cách giữa centroid
median	dựa trên median

Ví dụ

PHENIKAA UNIVERSITY
School of Computing

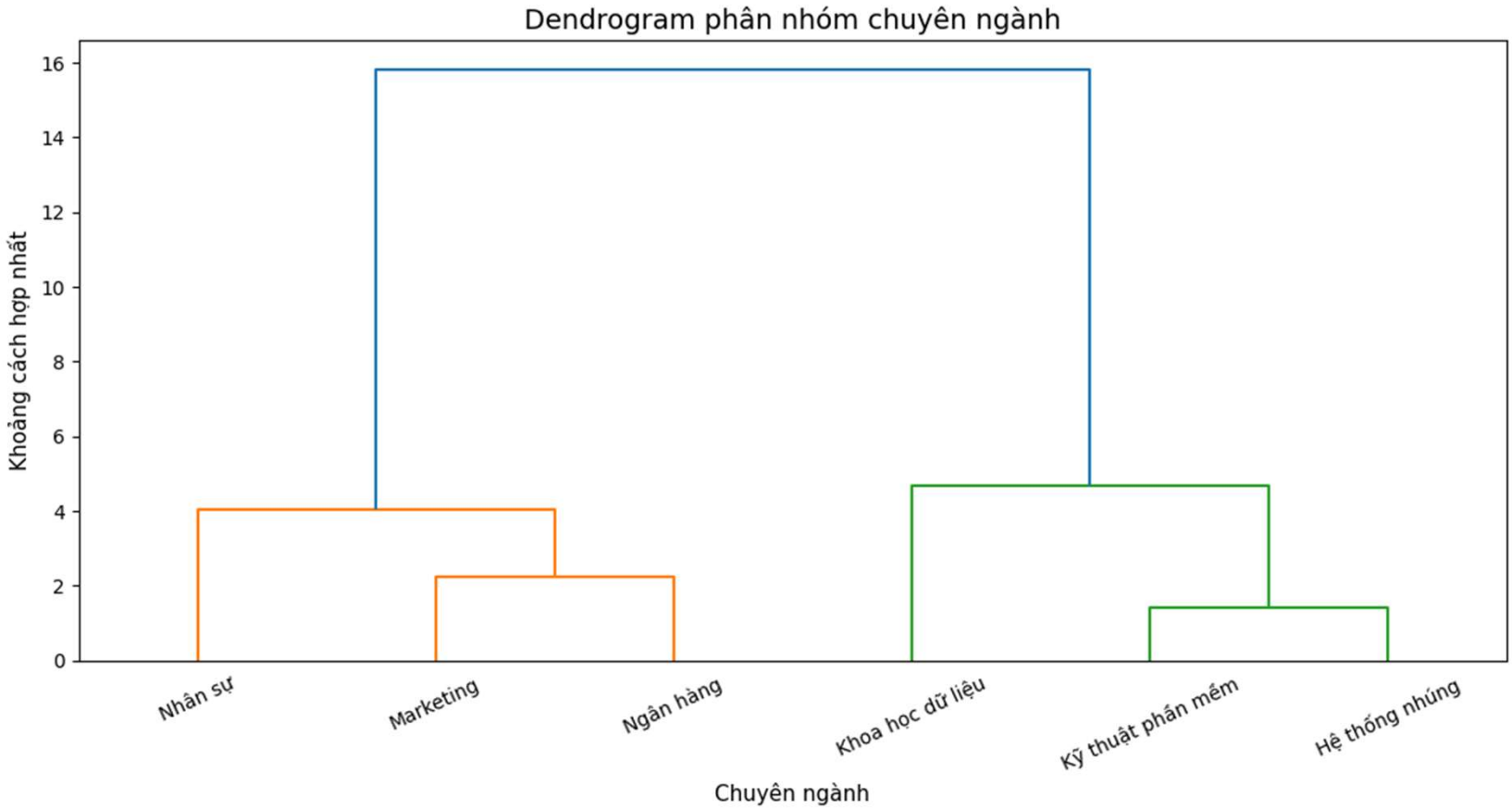
```
major_features = pd.DataFrame({
    "Chuyên ngành": ["Khoa học dữ liệu", "Kỹ thuật phần mềm",
                     "Hệ thống nhúng", "Marketing", "Nhân sự", "Ngân hàng"],
    "Lập trình": [9, 9, 8, 2, 1, 3],
    "Kinh doanh": [4, 2, 2, 9, 8, 9],
    "Phân tích": [10, 7, 6, 6, 4, 8]
}).set_index("Chuyên ngành")

Z = linkage(major_features.values, method="ward")
plt.figure(figsize=(11, 6))
dendrogram(Z, labels=major_features.index, leaf_rotation=25, leaf_font_size=10)
plt.title("Dendrogram phân nhóm chuyên ngành")
plt.xlabel("Chuyên ngành")
plt.ylabel("Khoảng cách hợp nhất")
plt.tight_layout()
plt.show()
```

Chuyên ngành	Lập trình	Kinh doanh	Phân tích
Khoa học dữ liệu	9	4	10
Kỹ thuật phần mềm	9	2	7
Hệ thống nhúng	8	2	6
Marketing	2	9	6
Nhân sự	1	8	4
Ngân hàng	3	9	8

Cách vẽ

PHENIKAA UNIVERSITY
School of Computing



Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

Treemap

Treemap biểu diễn cấu trúc phân cấp bằng các hình chữ nhật lồng nhau.

- **Diện tích** thường biểu diễn giá trị.
- **Màu sắc** có thể biểu diễn nhóm hoặc một biến định lượng khác.

Các hình chữ nhật lớn chứa các hình chữ nhật nhỏ hơn.



Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

Treemap

- **Phù hợp khi**
  - Cần hiển thị quan hệ phần và tổng thể.
  - Có nhiều nhóm và danh mục con.
  - Không gian hiển thị bị giới hạn.
  - Cần phát hiện nhanh nhóm có quy mô lớn.
- **Ưu điểm**
  - Sử dụng không gian hiệu quả.
  - Hiển thị được nhiều phần tử.
  - Dễ nhận ra giá trị rất lớn hoặc rất nhỏ.
  - Có thể biểu diễn đồng thời phân cấp, giá trị và màu sắc.
- **Hạn chế**
  - Khó so sánh chính xác diện tích của các hình có tỷ lệ gần nhau.
  - Các hình chữ nhật có hình dạng khác nhau làm giảm khả năng so sánh.
  - Quan hệ cha, con đôi khi không rõ.
  - Nhãn có thể bị cắt hoặc ẩn trong ô nhỏ.

Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- Sử dụng thư viện plotly: px.treemap()

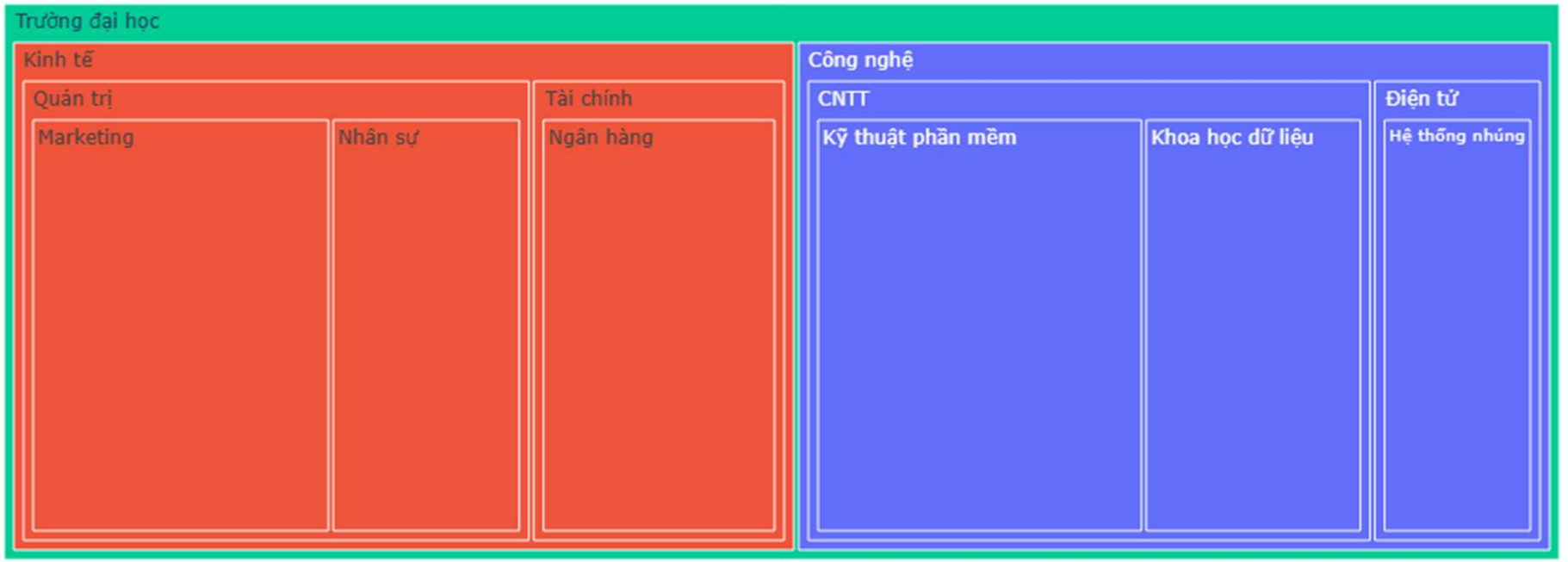
```
fig = px.treemap(
    hierarchy_df,
    path=[px.Constant("Trường đại học"), "Khoa", "Ngành", "Chuyên ngành"],
    values="Số sinh viên",
    color="Khoa",
    title="Treemap: Cơ cấu sinh viên theo khoa, ngành và chuyên ngành"
)
fig.update_traces(root_color="#f1f5f9")
fig.show()
```

Khoa	Ngành	Chuyên ngành	Số sinh viên
Công nghệ	CNTT	Khoa học dữ liệu	80
Công nghệ	CNTT	Kỹ thuật phần mềm	120
Công nghệ	Điện tử	Hệ thống nhúng	60
Kinh tế	Quản trị	Marketing	110
Kinh tế	Quản trị	Nhân sự	70
Kinh tế	Tài chính	Ngân hàng	90

Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

Treemap: Cơ cấu sinh viên theo khoa, ngành và chuyên ngành



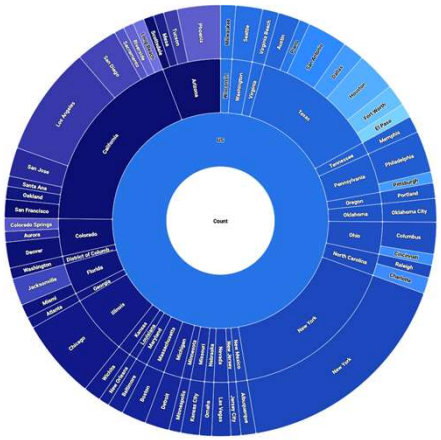
Các dạng biểu đồ trực quan phân cấp

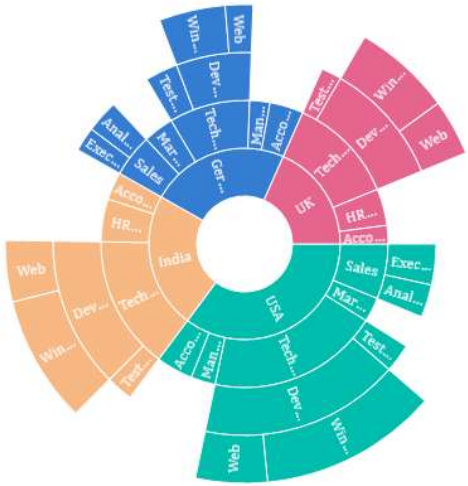
PHENIKAA UNIVERSITY
School of Computing

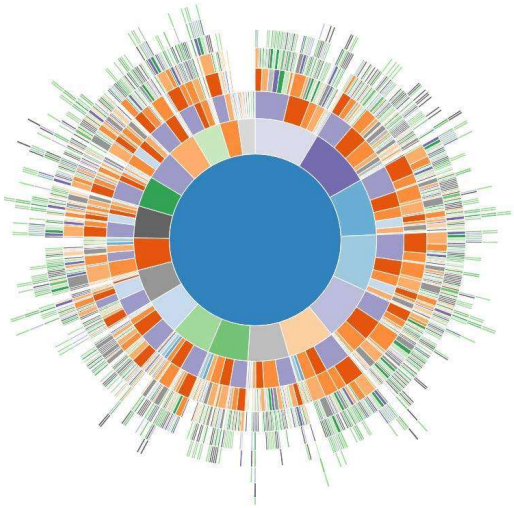
Sunburst

Sunburst sử dụng các vòng tròn đồng tâm để thể hiện cấu trúc phân cấp.

- Tâm của biểu đồ đại diện cho nút gốc.
- Mỗi vòng đại diện cho một cấp độ.
- Các cung tròn biểu diễn các nhóm và nhóm con.
- Góc hoặc diện tích cung thường thể hiện giá trị.









Các dạng biểu đồ trực quan phân cấp

PHENIKAA UNIVERSITY
School of Computing

Sunburst

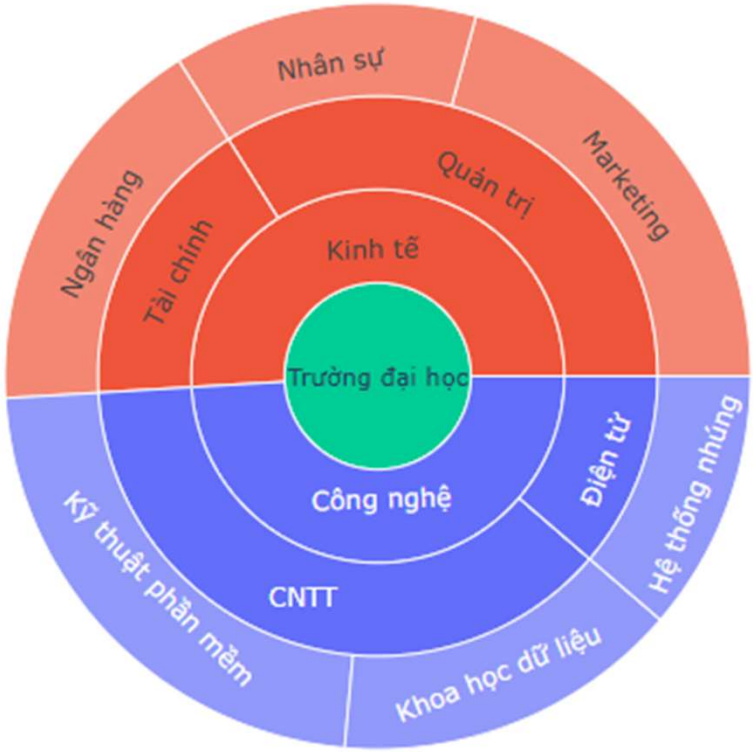
- **Phù hợp khi**
  - Muốn nhấn mạnh số cấp của cấu trúc.
  - Cần thể hiện đường đi từ cấp cao đến cấp thấp.
  - Cấu trúc không quá sâu.
  - Sản phẩm có tính tương tác để phóng to hoặc xem thông tin chi tiết.
- **Ưu điểm**
  - Thể hiện rõ các cấp độ.
  - Có hình thức trực quan hấp dẫn.
  - Người xem có thể theo dõi từ tâm ra ngoài.
- **Hạn chế**
  - Khó so sánh các cung không có cùng bán kính.
  - Nhãn ở vòng ngoài dễ bị chồng lấn.
  - Các lát nhỏ rất khó quan sát.
  - Không phù hợp khi cấu trúc quá sâu hoặc có quá nhiều nhánh.

Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

- Sử dụng thư viện plotly: px.sunburst()

```
fig = px.sunburst(
    hierarchy_df,
    path=[px.Constant("Trường đại học"), "Khoa", "Ngành", "Chuyên ngành"],
    values="Số sinh viên",
    color="Khoa",
    title="Sunburst: Các cấp trong cấu trúc chương trình"
)
fig.show()
```



So sánh các biểu đồ

PHENIKAA UNIVERSITY
School of Computing

Tiêu chí	Tree diagram	Treemap	Sunburst
Quan hệ cha, con	Rất rõ	Trung bình	Khá rõ
So sánh quy mô	Hạn chế	Khá tốt	Trung bình
Sử dụng không gian	Thấp	Cao	Trung bình
Thể hiện số cấp	Tốt	Trung bình	Tốt
Khả năng đọc nhãn	Tốt khi ít nút	Kém ở ô nhỏ	Kém ở cung nhỏ
Dữ liệu quy mô lớn	Dễ rối	Tương đối phù hợp	Dễ rối
Mức độ chính xác khi so sánh	Thấp	Trung bình	Thấp

Bài tập thực hành

PHENIKAA UNIVERSITY
School of Computing

Chuyển bảng sau thành cấu trúc phân cấp

- **Yêu cầu**
  - Xác định nút gốc.
  - Xác định các cấp phân cấp.
  - Vẽ cấu trúc bằng tree diagram.
  - Phác thảo cách biểu diễn bằng treemap.
  - Cho biết trường hợp nào nên dùng sunburst.
  - Chọn phương pháp phù hợp nhất nếu mục tiêu là so sánh số sinh viên.

Khoa	Ngành	Chuyên ngành	Số sinh viên
Công nghệ	CNTT	Khoa học dữ liệu	80
Công nghệ	CNTT	Kỹ thuật phần mềm	120
Công nghệ	Điện tử	Hệ thống nhúng	60
Kinh tế	Quản trị	Marketing	110
Kinh tế	Quản trị	Nhân sự	70
Kinh tế	Tài chính	Ngân hàng	90

2. Trực quan dữ liệu mạng

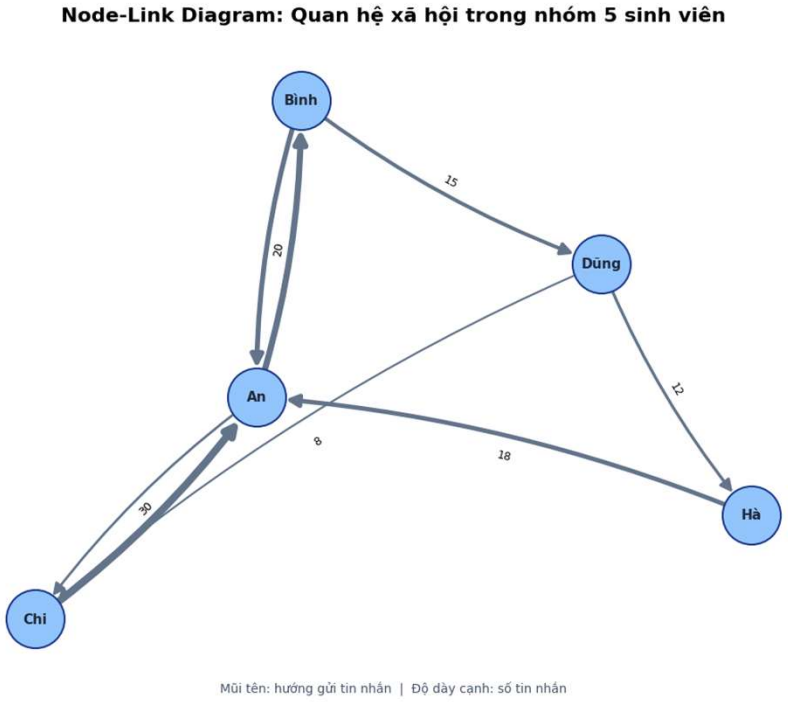
Khái niệm dữ liệu mạng

PHENIKAA UNIVERSITY
School of Computing

- Dữ liệu mạng mô tả các đối tượng và mối quan hệ giữa chúng.
- Một mạng có thể biểu diễn dưới dạng:

$$G = (V, E)$$

- Trong đó:
  - V: tập hợp các nút.
  - E: tập hợp các cạnh kết nối các nút.

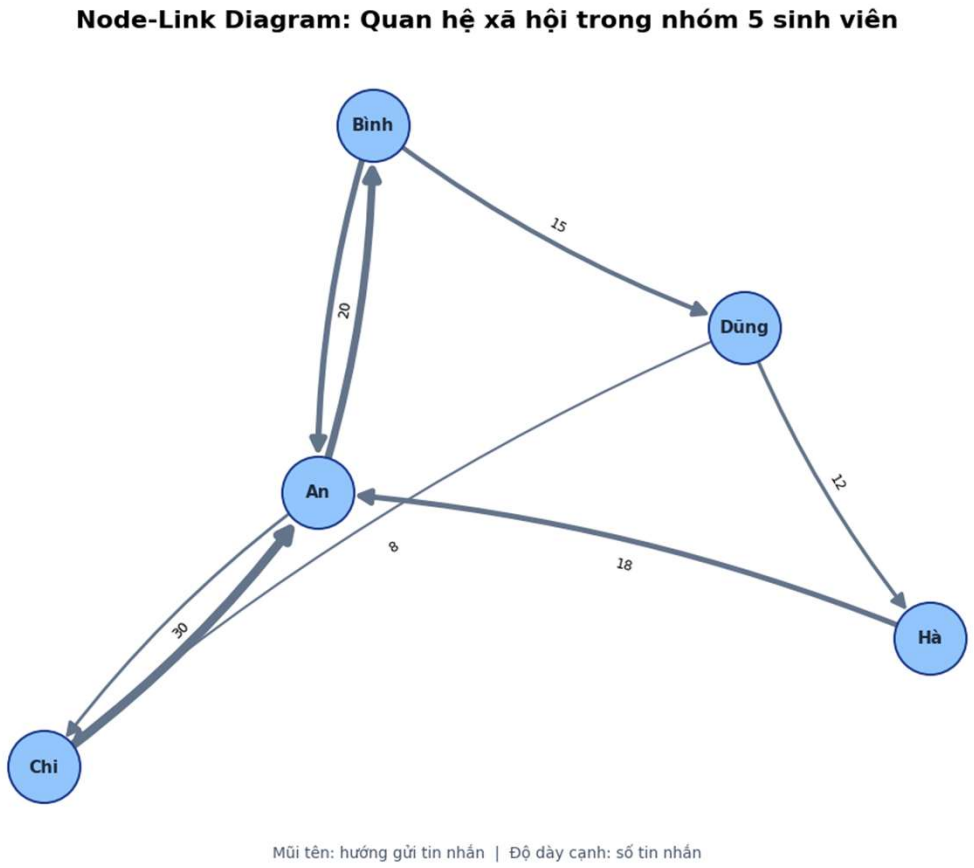


Các thành phần của mạng

PHENIKAA UNIVERSITY
School of Computing

1. Nút

- Nút đại diện cho một đối tượng.
- Ví dụ:
  - Người dùng trong mạng xã hội.
  - Thành phố trong mạng giao thông.
  - Máy tính trong mạng nội bộ.
  - Bài báo trong mạng trích dẫn.
  - Sản phẩm trong mạng mua chung.

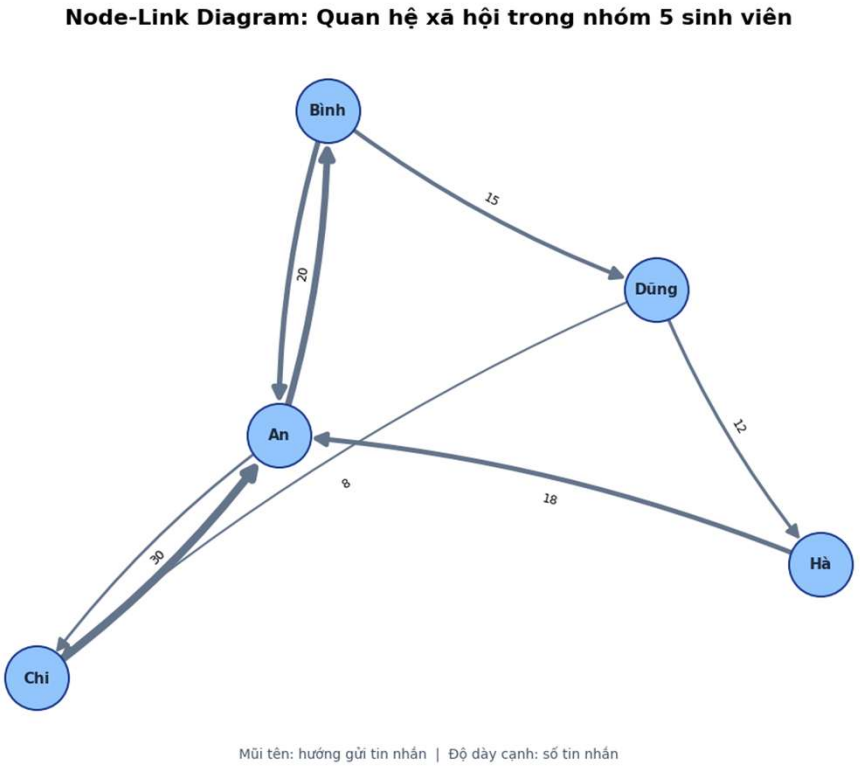


Các thành phần của mạng

PHENIKAA UNIVERSITY
School of Computing

2. Cạnh

- Cạnh biểu diễn quan hệ giữa hai nút.
- Ví dụ:
  - Quan hệ bạn bè.
  - Quan hệ hợp tác.
  - Đường bay giữa hai thành phố.
  - Một bài báo trích dẫn bài báo khác.
  - Người dùng gửi tin nhắn cho nhau.

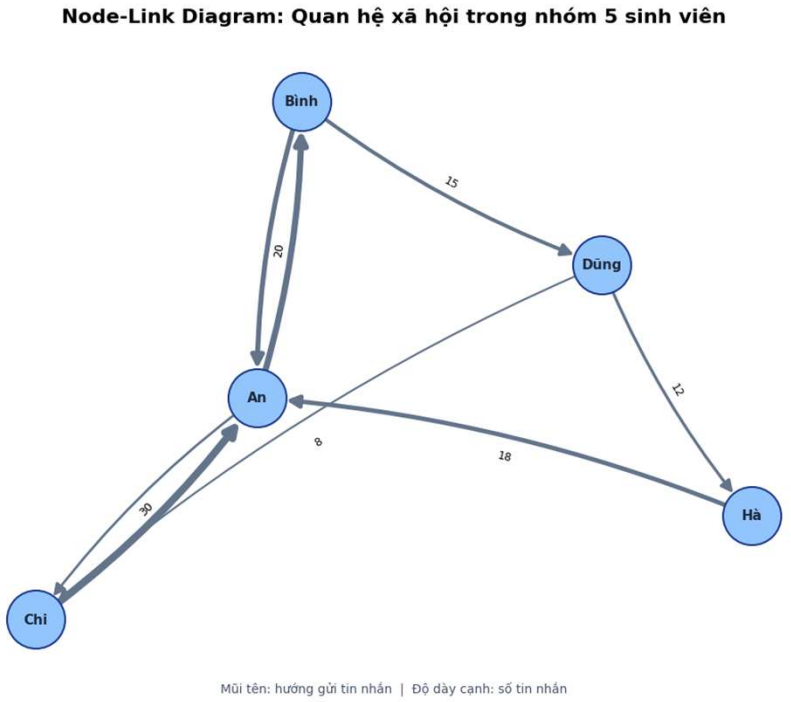


Các thành phần của mạng

PHENIKAA UNIVERSITY
School of Computing

3. Hướng

- **Mạng vô hướng**
  - Quan hệ không có chiều.
  - Ví dụ: A và B là bạn bè.
  - A —— B
- **Mạng có hướng**
  - Quan hệ có chiều từ nút nguồn đến nút đích.
  - Ví dụ: A theo dõi B.
  - A ——▶ B
  - Việc A theo dõi B không có nghĩa là B theo dõi A.

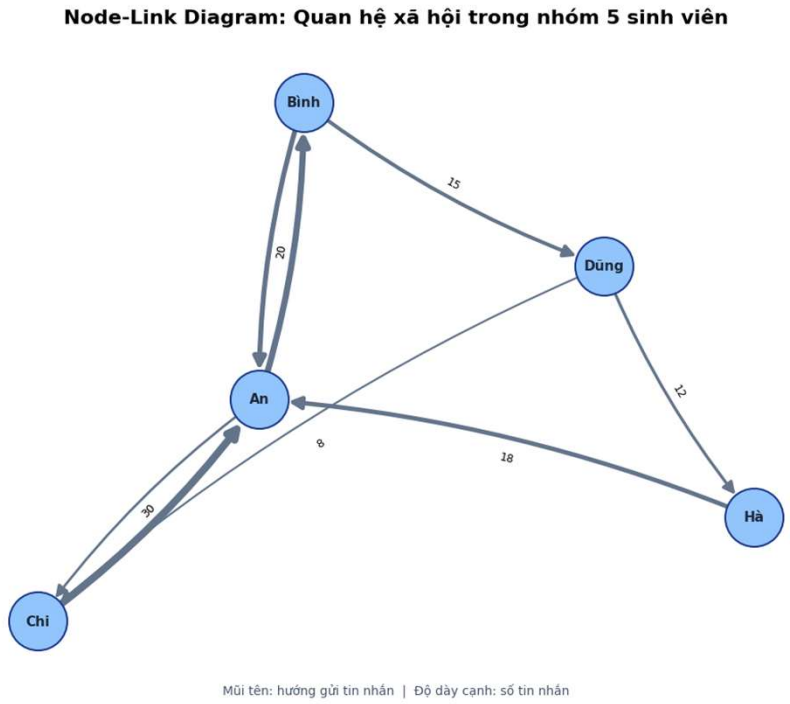


Các thành phần của mạng

PHENIKAA UNIVERSITY
School of Computing

4. Trọng số

- Trọng số cho biết cường độ hoặc quy mô của mối quan hệ.
- Ví dụ:
  - Số tin nhắn được gửi.
  - Số lần hợp tác.
  - Số lượt di chuyển.
  - Giá trị giao dịch.
  - Khoảng cách giữa hai điểm.

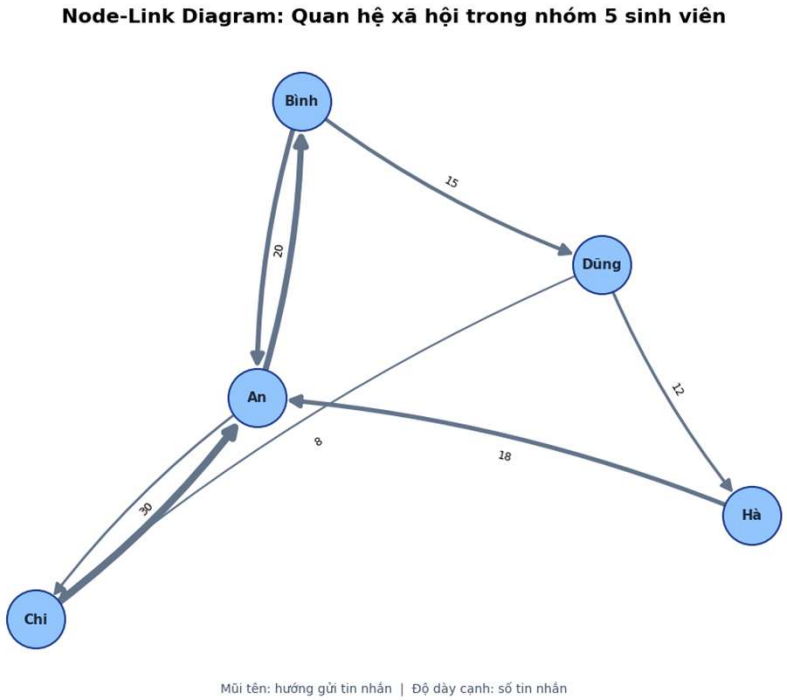


Các thành phần của mạng

PHENIKAA UNIVERSITY
School of Computing

5. Bậc của nút

- Bậc của nút là số cạnh nối với nút đó.
- Trong mạng có hướng
  - **In-degree:** Số cạnh đi vào nút.
  - **Out-degree:** Số cạnh đi ra khỏi nút.
- Ví dụ
  - Nếu A nhận liên kết từ B, C và D thì: $(A) = 3$
  - Nếu A tạo liên kết đến E và F thì: $(A) = 2$



Các thành phần của mạng

PHENIKAA UNIVERSITY
School of Computing

5. Bậc của nút

- **Ý nghĩa:** Nút có bậc cao có thể là:
  - Trung tâm giao tiếp.
  - Điểm kết nối quan trọng.
  - Đối tượng có ảnh hưởng.
  - Điểm có nguy cơ gây tắc nghẽn.

!!! bậc cao ≠ mức độ quan trọng cao.

Tầm quan trọng còn phụ thuộc vào loại quan hệ và cấu trúc toàn mạng.

PHENIKAA UNIVERSITY
School of Computing

Node-link diagram

Node-link diagram sử dụng các hình tròn hoặc ký hiệu làm nút, đường nối làm cạnh.

- **Kênh thị giác thường dùng**
  - Kích thước nút: bậc, doanh thu hoặc mức độ ảnh hưởng.
  - Màu nút: nhóm hoặc cộng đồng.
  - Độ dày cạnh: trọng số.
  - Mũi tên: hướng.
  - Độ trong suốt: giảm chồng lấn.
  - Vị trí: kết quả của thuật toán bố cục.

Biểu diễn mạng

PHENIKAA UNIVERSITY
School of Computing

Node-link diagram

- **Ưu điểm**
  - Trực quan và dễ giải thích.
  - Dễ theo dõi đường đi.
  - Dễ nhận biết nút trung tâm.
  - Có thể quan sát cụm hoặc cộng đồng.
- **Hạn chế**
  - Nhiều cạnh có thể tạo thành “búi tóc”.
  - Vị trí nút có thể thay đổi theo thuật toán.
  - Khoảng cách trên màn hình không phải lúc nào cũng có ý nghĩa.
  - Khó so sánh chính xác số cạnh.
  - Dễ tạo ra kết luận sai nếu người xem cho rằng nút ở giữa luôn quan trọng nhất.

Cách vẽ

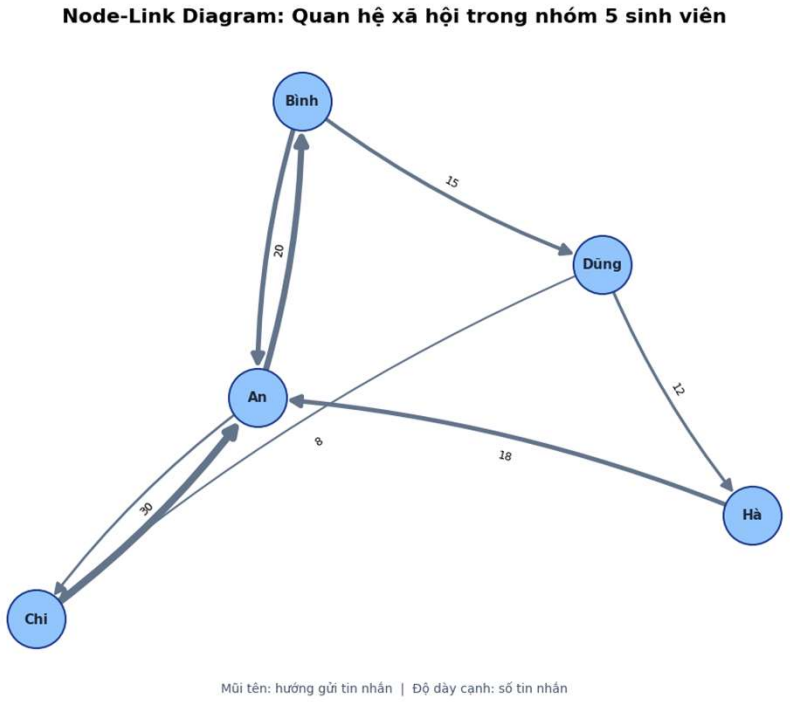
PHENIKAA UNIVERSITY
School of Computing

- Chuyển dữ liệu thành graph

```
import networkx as nx

G = nx.DiGraph()

G.add_edge("An", "Bình", weight=25)
G.add_edge("An", "Chi", weight=10)
G.add_edge("Bình", "An", weight=20)
G.add_edge("Bình", "Dũng", weight=15)
G.add_edge("Chi", "An", weight=30)
G.add_edge("Dũng", "Chi", weight=8)
```



Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

Chọn layout: *pos = nx.<layout_name>(G, seed=42)*

- **Spring layout**
  - Các node được xem như những vật thể có lực đẩy–hút
  - pos = nx.spring_layout(G, seed=42)
  - Phù hợp với mạng xã hội, mạng cộng tác...
- **Circular layout**
  - Các node nằm trên vòng tròn
  - pos = nx.circular_layout(G)
  - Phù hợp khi có ít node
- **…**

Các layout

PHENIKAA UNIVERSITY
School of Computing

Layout	Cách bố trí	Ứng dụng điển hình
spring_layout()	Lực hút–đẩy	Social network
circular_layout()	Vòng tròn	Network nhỏ
random_layout()	Ngẫu nhiên	Thử nghiệm
shell_layout()	Vòng đồng tâm	Network nhiều tầng
spectral_layout()	Spectral graph	Phân tích cấu trúc
kamada_kawai_layout()	Khoảng cách graph	Network nhỏ/vừa
planar_layout()	Hạn chế giao cạnh	Planar graph
bipartite_layout()	Hai phía	User–item
multipartite_layout()	Nhiều tầng	Input→Output, workflow
spiral_layout()	Xoắn ốc	Trường hợp đặc biệt
Custom pos	Tự định nghĩa	Sơ đồ cần kiểm soát

Cách vẽ

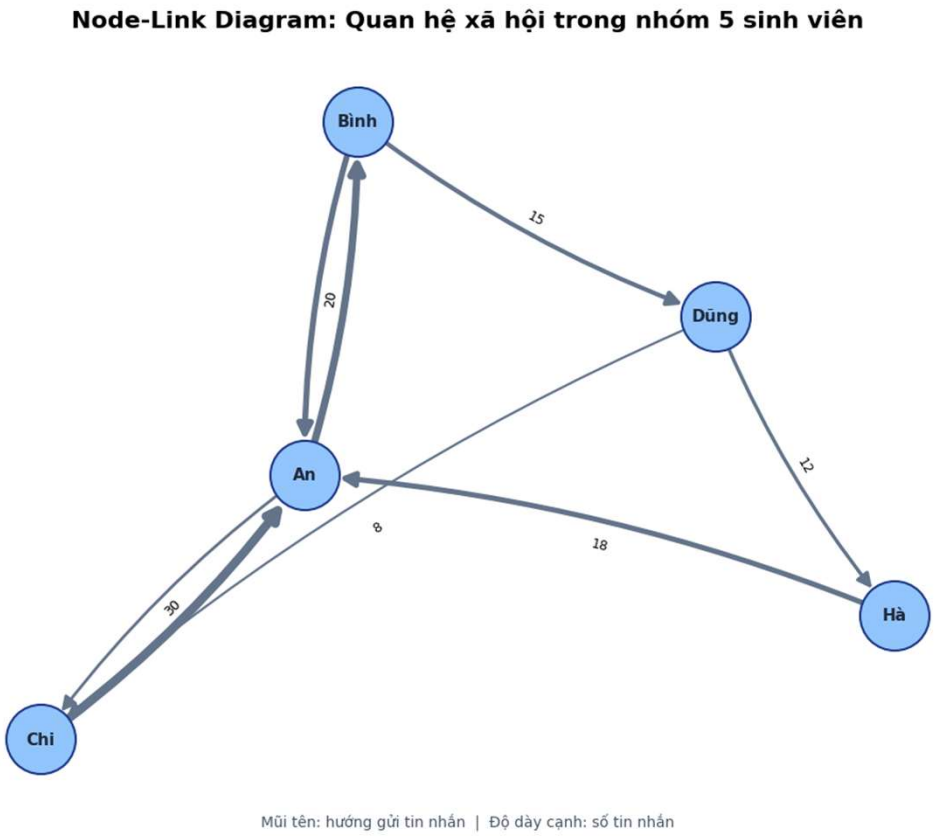
PHENIKAA UNIVERSITY
School of Computing

- Vẽ node

```
nx.draw_networkx_nodes(
    G,
    pos,
    node_size=1200,
    node_color="lightblue"
)
```

- Vẽ cạnh

```
nx.draw_networkx_edges(
    G,
    pos,
    arrows=True,
    arrowstyle="->"
)
```



Cách vẽ

PHENIKAA UNIVERSITY
School of Computing

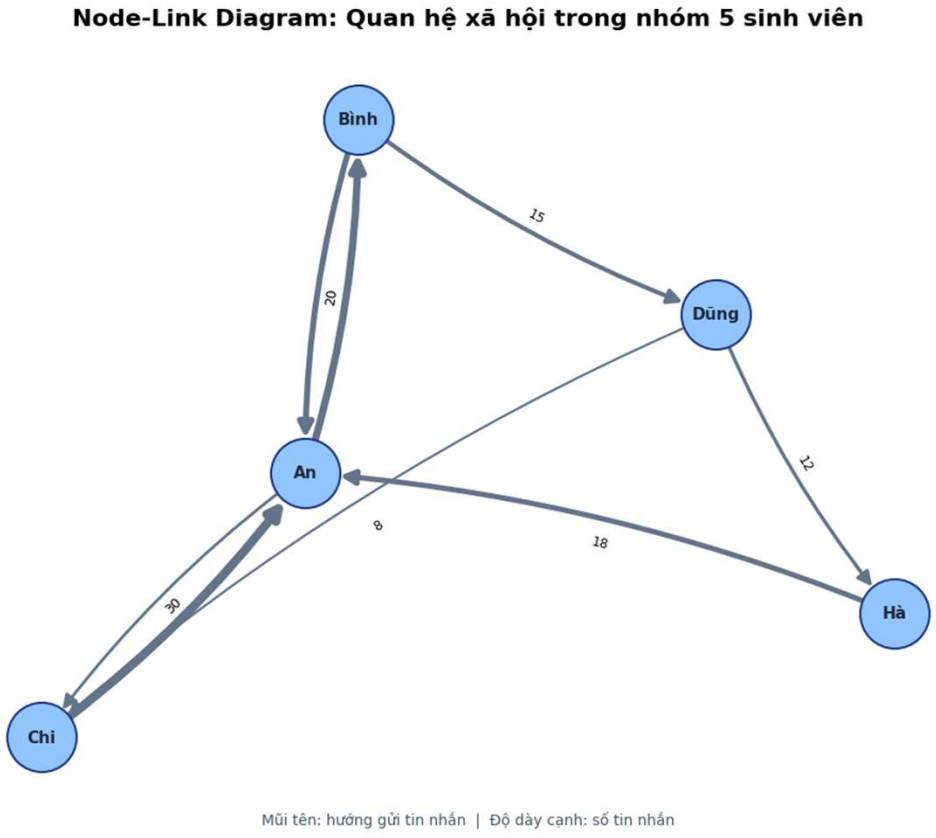
- Vẽ nhãn node

```
nx.draw_networkx_labels(
    G,
    pos,
    font_size=11
)
```

- Vẽ nhãn cạnh

```
edge_labels = nx.get_edge_attributes(G, "weight")

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels
)
```



Biểu diễn mạng

PHENIKAA UNIVERSITY
School of Computing

Adjacency matrix

Adjacency matrix sử dụng bảng gồm hàng và cột đại diện cho các nút.

Nếu nút hàng có quan hệ với nút cột, ô tương ứng được đánh dấu hoặc tô màu.

- **Ví dụ**

Trong đó:

- 1: Có liên kết.
- 0: Không có liên kết.

	A	**B**	**C**	**D**
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Nếu mạng có trọng số, các ô có thể chứa giá trị trọng số thay vì chỉ chứa 0 hoặc 1.

Biểu diễn mạng

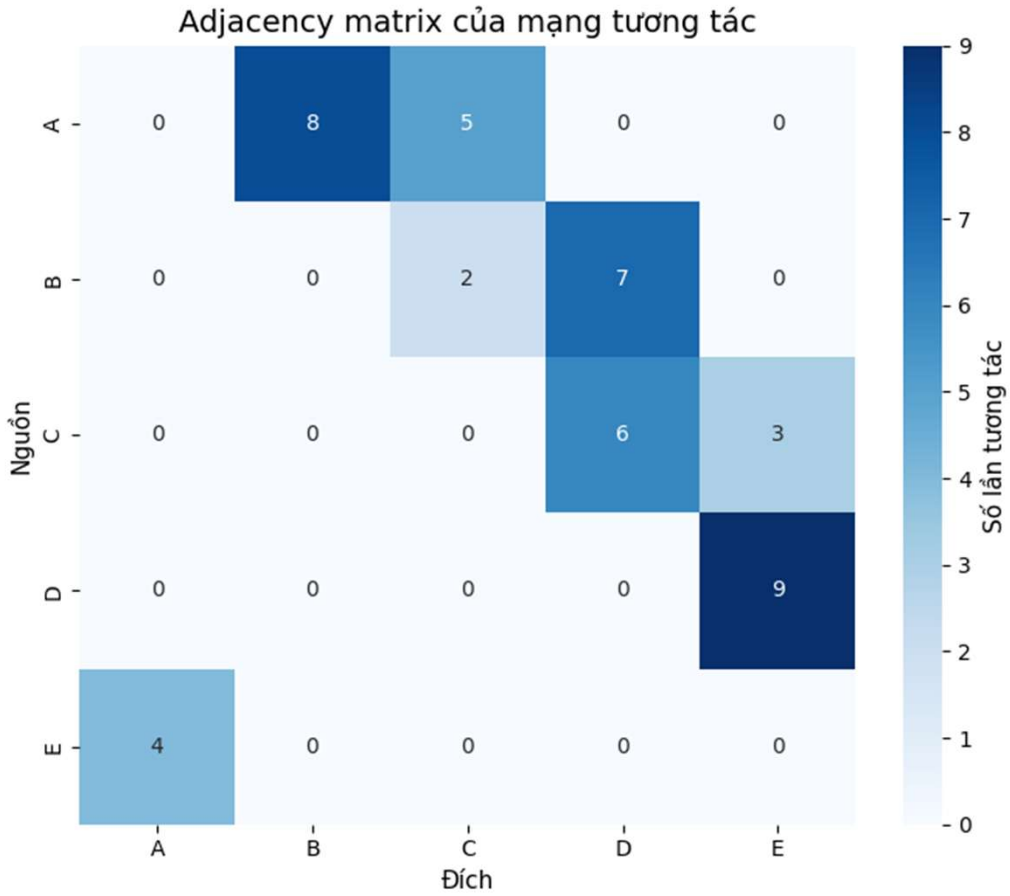
PHENIKAA UNIVERSITY
School of Computing

Adjacency matrix

- **Ưu điểm**
  - Không có hiện tượng cạnh chồng lên nhau.
  - Phù hợp hơn khi mạng có mật độ cao.
  - Dễ phát hiện các khối hoặc cụm nếu sắp xếp hàng, cột hợp lý.
  - Có thể biểu diễn trọng số bằng màu sắc.
- **Hạn chế**
  - Khó theo dõi đường đi giữa nhiều nút.
  - Kém trực quan đối với người mới.
  - Kết quả phụ thuộc mạnh vào thứ tự hàng và cột.
  - Với mạng thưa, ma trận có nhiều khoảng trống.

Dùng heatmap trực quan adjacency matrix

PHENIKAA UNIVERSITY
School of Computing



So sánh 2 cách biểu diễn

PHENIKAA UNIVERSITY
School of Computing

Tiêu chí	Node-link diagram	Adjacency matrix
Theo dõi đường đi	*Tốt*	*Khó*
Nhận biết nút trung tâm	*Tốt khi mạng nhỏ*	*Cần tính hoặc sắp xếp*
Mạng dày đặc	*Dễ rối*	*Phù hợp hơn*
Phát hiện cụm	*Khá tốt*	*Tốt nếu sắp xếp hợp lý*
Quan sát liên kết trực tiếp	*Trực quan*	*Cần dò hàng và cột*
Hiện tượng chồng lấn	*Cao*	*Không có*
Phụ thuộc bố cục	*Cao*	*Phụ thuộc thứ tự hàng, cột*

Khi nào không nên dung note-link diagram

PHENIKAA UNIVERSITY
School of Computing

- Không nên sử dụng node-link diagram khi:
  - Có quá nhiều nút và cạnh.
  - Mục tiêu chỉ là xếp hạng các đối tượng.
  - Người xem cần so sánh chính xác giá trị.
  - Cấu trúc quan hệ không phải phần quan trọng của câu hỏi.
  - Dữ liệu không có quan hệ thực sự giữa các phần tử.
  - Sơ đồ chỉ tạo cảm giác phức tạp nhưng không cung cấp thêm thông tin.
  - Không thể giải thích ý nghĩa của vị trí, kích thước, màu sắc và đường nối.

Khi nào không nên dung note-link diagram

PHENIKAA UNIVERSITY
School of Computing

- **Giải pháp thay thế**

Mục tiêu	Dạng trực quan thay thế
Xếp hạng nút	Biểu đồ thanh
So sánh số liên kết	Biểu đồ thanh theo bậc
Mạng dày đặc	Adjacency matrix
So sánh nhóm	Heatmap
Quan sát phân phối bậc	Histogram
Theo dõi luồng giữa nhóm	Sankey diagram
Thể hiện quan hệ theo thời gian	Timeline hoặc biểu đồ đường

Bài tập thực hành

PHENIKAA UNIVERSITY
School of Computing

Vẽ đồ thị mạng

- **Yêu cầu**
  - Các cột nào đại diện cho nút?
  - Mạng có hướng hay vô hướng?
  - Trọng số là biến nào?
  - Nút nào có nhiều liên kết?
  - Có thể xây dựng adjacency matrix như thế nào?
  - Nếu dữ liệu tăng lên 1.000 nút và 30.000 cạnh, node-link diagram có còn phù hợp không?
  - Đề xuất phương án thay thế.

Nguồn	Đích	Số lần tương tác
A	B	8
A	C	5
B	C	2
B	D	7
C	D	6
C	E	3
D	E	9
E	A	4

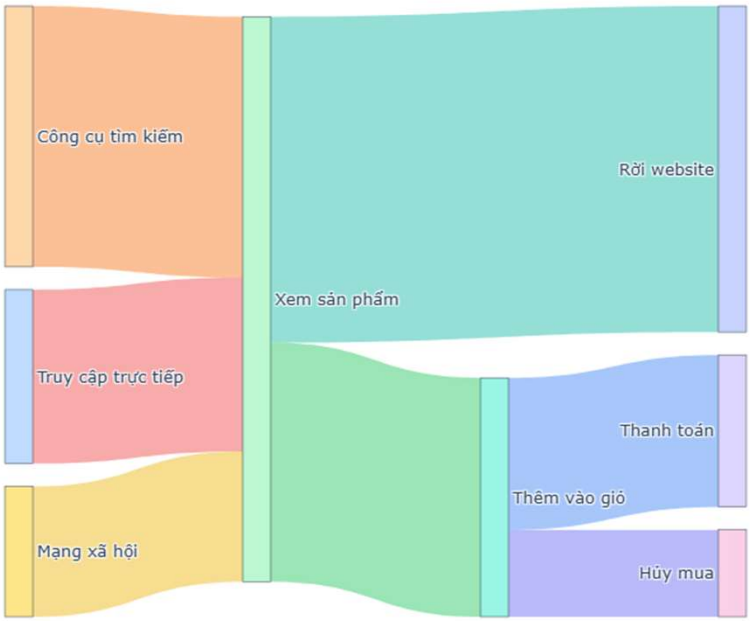
3. Trực quan dữ liệu luồng

Khái niệm dữ liệu luồng

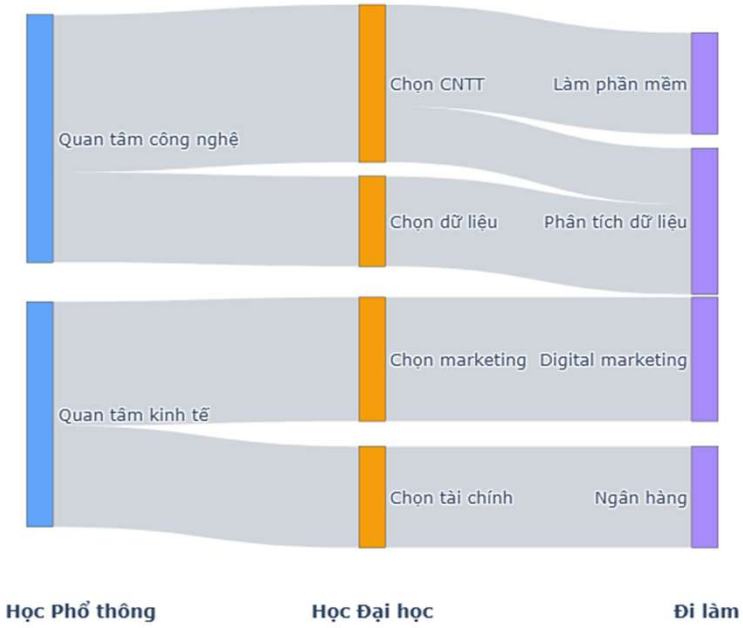
PHENIKAA UNIVERSITY
School of Computing

- Dữ liệu luồng mô tả sự di chuyển, truyền dẫn hoặc chuyển đổi giữa các trạng thái, địa điểm hoặc nhóm.

Sankey: Hành trình người dùng trên website



Alluvial Diagram: Hành trình lựa chọn nghề nghiệp



Thành phần cơ bản

PHENIKAA UNIVERSITY
School of Computing

1. Nguồn

- Điểm bắt đầu của luồng.
- Ví dụ:
  - Kênh quảng cáo.
  - Quốc gia xuất khẩu.
  - Trạng thái ban đầu của khách hàng.
  - Loại năng lượng đầu vào.

Nguồn	Đích	Giá trị
Quảng cáo	Trang sản phẩm	500
Tìm kiếm	Trang sản phẩm	800
Mạng xã hội	Trang sản phẩm	300
Trang sản phẩm	Giỏ hàng	700
Trang sản phẩm	Rời trang	900
Giỏ hàng	Thanh toán	450
Giỏ hàng	Hủy	250

Thành phần cơ bản

PHENIKAA UNIVERSITY
School of Computing

2. Đích

- Điểm kết thúc hoặc trạng thái tiếp theo.
- Ví dụ:
  - Trang đích.
  - Quốc gia nhập khẩu.
  - Trạng thái mua hàng.
  - Ngành sử dụng năng lượng.

Nguồn	Đích	Giá trị
Quảng cáo	Trang sản phẩm	500
Tìm kiếm	Trang sản phẩm	800
Mạng xã hội	Trang sản phẩm	300
Trang sản phẩm	Giỏ hàng	700
Trang sản phẩm	Rời trang	900
Giỏ hàng	Thanh toán	450
Giỏ hàng	Hủy	250

Thành phần cơ bản

PHENIKAA UNIVERSITY
School of Computing

3. Khối lượng luồng

- Cho biết quy mô của sự di chuyển.
- Ví dụ:
  - Số người dùng.
  - Số tiền.
  - Số sản phẩm.
  - Khối lượng hàng hóa.
  - Lượng năng lượng.
  - Số sinh viên chuyển ngành.

Nguồn	Đích	Giá trị
Quảng cáo	Trang sản phẩm	500
Tìm kiếm	Trang sản phẩm	800
Mạng xã hội	Trang sản phẩm	300
Trang sản phẩm	Giỏ hàng	700
Trang sản phẩm	Rời trang	900
Giỏ hàng	Thanh toán	450
Giỏ hàng	Hủy	250

Trực quan dữ liệu luồng

PHENIKAA UNIVERSITY
School of Computing

Sankey diagram

Sankey diagram sử dụng các dải nối giữa các nút. Độ rộng của dải biểu diễn khối lượng luồng.

- **Câu hỏi Sankey có thể trả lời**
  - Nguồn nào tạo ra luồng lớn nhất?
  - Luồng bị thất thoát ở bước nào?
  - Đích nào tiếp nhận nhiều nhất?
  - Nhóm người dùng chuyển sang trạng thái nào?
  - Có nút trung gian nào đóng vai trò quan trọng?



Trực quan dữ liệu luồng

PHENIKAA UNIVERSITY
School of Computing

Sankey diagram

- **Ưu điểm**
  - Thể hiện rõ nguồn, đích và khối lượng.
  - Hữu ích khi phân tích quá trình nhiều bước.
  - Dễ nhìn thấy điểm thất thoát.
  - Tạo ra câu chuyện trực quan về sự chuyển đổi.
- **Hạn chế**
  - Khó so sánh các dải có độ rộng gần nhau.
  - Nhiều liên kết gây chồng chéo.
  - Người xem có thể khó theo dõi một dải qua nhiều giai đoạn.
  - Thứ tự nút ảnh hưởng đến số đoạn giao nhau.
  - Không phù hợp khi cần đọc giá trị chính xác.

Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Sử dụng thư viện plotly

```
pip install plotly
```

```
import plotly.graph_objects as go
```

Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Chuẩn bị dữ liệu

```
import pandas as pd

flow_df = pd.DataFrame({
    "Nguồn": [
        "Trang chủ", "Trang chủ", "Sản phẩm", "Sản phẩm", "Blog"
    ],
    "Đích": [
        "Sản phẩm", "Blog", "Mua hàng", "Thoát", "Thoát"
    ],
    "Giá trị": [
        500, 300, 350, 150, 300
    ]
})
flow_df
```

	Nguồn	Đích	Giá trị
0	Trang chủ	Sản phẩm	500
1	Trang chủ	Blog	300
2	Sản phẩm	Mua hàng	350
3	Sản phẩm	Thoát	150
4	Blog	Thoát	300

Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Tạo danh sách các node

```
['Trang chủ', 'Sản phẩm', 'Blog', 'Mua hàng', 'Thoát']
```

```
labels = pd.Index(
    pd.unique(
        flow_df[["Nguồn", "Đích"]].values.ravel("K")
    )
).tolist()
```

```
label_to_id = {
    label: i
    for i, label in enumerate(labels)
}
```

```
Trang chủ → 0
Sản phẩm  → 1
Blog      → 2
Mua hàng  → 3
Thoát     → 4
```

Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Chuyển source và target thành id

```
source_ids = flow_df["Nguồn"].map(label_to_id)
target_ids = flow_df["Đích"].map(label_to_id)
```

```
Nguồn:
Trang chủ → 0
Trang chủ → 0
Sản phẩm  → 1
Sản phẩm  → 1
Blog      → 2
```

```
Đích:
Sản phẩm → 1
Blog     → 2
Mua hàng → 3
Thoát    → 4
Thoát    → 4
```

Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Vẽ bằng hàm go.Sankey()

```
fig = go.Figure(
    go.Sankey(
        node=dict(
            label=labels, pad=20, thickness=25,
            line=dict(
                color="black",
                width=0.5
            )
        ),

        link=dict(
            source=source_ids, target=target_ids, value=flow_df["Giá trị"]
        )
    )
)
```



Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Đặt tiêu đề, cỡ chữ, kích thước biểu đồ

```
fig.update_layout(
    title="Sankey: Hành trình người dùng trên website",
    font_size=12,
    width=900,
    height=500
)
```

Cách vẽ Sankey Diagram

PHENIKAA UNIVERSITY
School of Computing

- Tô màu cho các luồng

```
link_colors = [
    "rgba(59, 130, 246, 0.45)",
    "rgba(16, 185, 129, 0.45)",
    "rgba(245, 158, 11, 0.45)",
    "rgba(239, 68, 68, 0.45)",
    "rgba(139, 92, 246, 0.45)"
]
```

```
fig = go.Figure(
    go.Sankey(
        node=dict(
            label=labels, pad=20, thickness=25
        ),

        link=dict(
            source=source_ids, target=target_ids, value=flow_df["Giá trị"],
            color=link_colors
        )
    )
)
```



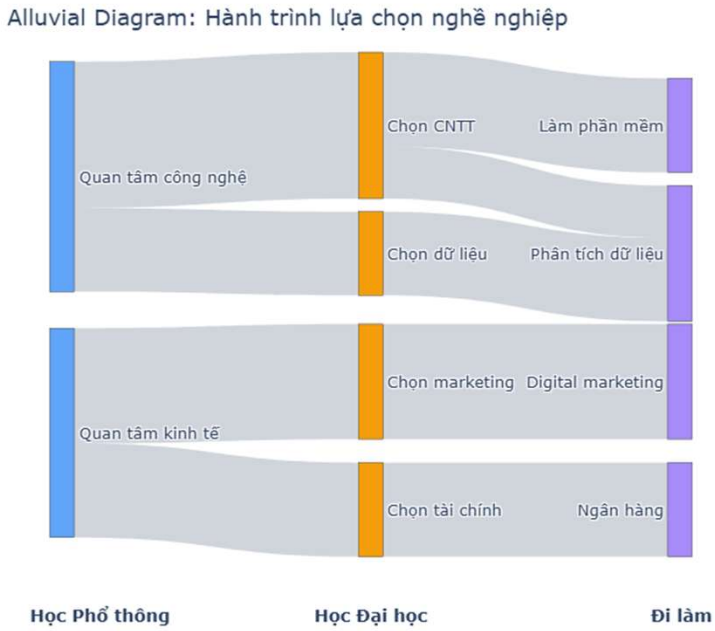
Trực quan dữ liệu luồng

PHENIKAA UNIVERSITY
School of Computing

Alluvial diagram

Alluvial diagram thể hiện cách các nhóm thay đổi thành phần qua nhiều giai đoạn hoặc thời điểm.

- **Ví dụ:** Theo dõi sinh viên qua ba giai đoạn:
  - Phổ thông, Đại học, Đi làm
  - Quan tâm CNTT
    - → Chọn CNTT → Làm phần mềm
    - → Chọn dữ liệu → Phân tích dữ liệu
  - Quan tâm kinh tế
    - → Chọn marketing → Digital marketing
    - → Chọn tài chính → Ngân hàng



Alluvial diagram

- **Điểm khác biệt với Sankey**
  - Sankey thường nhấn mạnh **khối lượng luồng giữa nguồn và đích**.
  - Alluvial thường nhấn mạnh **sự thay đổi nhóm qua các giai đoạn**.
  - Trong thực tế, hai thuật ngữ đôi khi được sử dụng gần giống nhau.
- **Trường hợp sử dụng**
  - Thay đổi nghề nghiệp qua thời gian.
  - Chuyển đổi nhóm khách hàng.
  - Thay đổi lựa chọn ngành học.
  - Dòng bỏ phiếu giữa các kỳ khảo sát.
  - Chuyển trạng thái của sản phẩm hoặc đơn hàng.

Cách vẽ Alluvial Diagram

PHENIKAA UNIVERSITY
School of Computing

- Tương tự Sankey Diagram – Dùng hàm go.Sankey()
- Thêm các nhãn giai đoạn bằng go.Figure().add_annotation()

```
fig.add_annotation(
    x=0.0,
    y=-0.08,
    xref="paper",
    yref="paper",
    text="<b>Học Phổ thông</b>",
    showarrow=False,
    font=dict(size=14)
)
```



Bài tập thực hành

PHENIKAA UNIVERSITY
School of Computing

Xây dựng biểu đồ Sankey cho dữ liệu

- **Yêu cầu:**
  - Xác định các nút nguồn.
  - Xác định các nút trung gian.
  - Xác định các nút đích.
  - Xây dựng Sankey diagram.
  - Viết ba nhận xét về hành trình người dùng.
  - Đề xuất một cải tiến thiết kế biểu đồ.

Nguồn	Đích	Giá trị
Truy cập trực tiếp	Xem sản phẩm	400
Công cụ tìm kiếm	Xem sản phẩm	600
Mạng xã hội	Xem sản phẩm	300
Xem sản phẩm	Thêm vào giỏ	550
Xem sản phẩm	Rời website	750
Thêm vào giỏ	Thanh toán	350
Thêm vào giỏ	Hủy mua	200

4. Một số lưu ý chọn và vẽ biểu đồ

CÁC VẤN ĐỀ VỀ KHẢ NĂNG ĐỌC

PHENIKAA UNIVERSITY
School of Computing

- **Chồng lấn**
  - Chồng lấn xuất hiện khi:
  - Nhiều cạnh cắt nhau.
  - Nhiều nhãn nằm gần nhau.
  - Nút hoặc dải luồng che khuất nhau.
  - Các thành phần quá nhỏ.
- **Giải pháp**
  - Lọc dữ liệu.
  - Tổng hợp các nhóm nhỏ.
  - Sử dụng độ trong suốt.
  - Chỉ hiển thị nhãn quan trọng.
  - Cho phép tương tác khi rê chuột hoặc chọn nút.
  - Chia trực quan thành nhiều hình nhỏ.
  - Sử dụng adjacency matrix đối với mạng dày đặc.

CÁC VẤN ĐỀ VỀ KHẢ NĂNG ĐỌC

PHENIKAA UNIVERSITY
School of Computing

- **Khó so sánh chính xác**
  - Con người so sánh độ dài trên cùng một trục tốt hơn so sánh diện tích, góc, cung tròn, hay độ rộng của các dải ở vị trí khác nhau.
- **Vì vậy:**
  - Treemap phù hợp để thấy tổng quan nhưng không tốt cho so sánh gần đúng.
  - Sunburst thể hiện cấp độ tốt nhưng khó so sánh giá trị.
  - Sankey cho thấy luồng lớn, nhỏ nhưng không phù hợp để đọc số chính xác.
  - Nếu yêu cầu so sánh chính xác, nên bổ sung biểu đồ thanh hoặc bảng.

CÁC VẤN ĐỀ VỀ KHẢ NĂNG ĐỌC

PHENIKAA UNIVERSITY
School of Computing

- **Quá nhiều nút và cạnh**
  - Một biểu đồ không trở nên hữu ích chỉ vì nó hiển thị được toàn bộ dữ liệu.
  - Khi số nút và cạnh tăng:
    - Khả năng đọc giảm.
    - Nhãn biến mất.
    - Đường nối chồng lên nhau.
  - Người xem không biết nên tập trung vào đâu.
  - Các mẫu quan trọng có thể bị che khuất.
- **Chiến lược xử lý**
  - Lọc theo thời gian, nhóm hoặc trọng số.
  - Loại bỏ liên kết không cần thiết.
  - Tổng hợp nút theo cộng đồng.
  - Hiển thị mạng con.
  - Cho phép người dùng tìm kiếm.
  - Làm nổi bật một đường đi hoặc một nhóm tại một thời điểm.
  - Kết hợp nhiều biểu đồ thay vì ép mọi thông tin vào một hình.

CÁC VẤN ĐỀ VỀ KHẢ NĂNG ĐỌC

PHENIKAA UNIVERSITY
School of Computing

- **Node-link - Bố cục làm thay đổi cảm nhận**
  - Hai node-link diagram có thể thể hiện cùng một dữ liệu nhưng tạo cảm giác khác nhau nếu sử dụng bố cục khác nhau.

CÁC VẤN ĐỀ VỀ KHẢ NĂNG ĐỌC

PHENIKAA UNIVERSITY
School of Computing

- **Các loại bố cục**
  - Bố cục lực đẩy có thể làm các cụm tách xa nhau.
  - Bố cục vòng tròn nhấn mạnh sự bình đẳng giữa các nút.
  - Bố cục phân cấp nhấn mạnh hướng hoặc cấp bậc.
  - Bố cục theo tọa độ địa lý nhấn mạnh vị trí thật.
  - Vị trí ở trung tâm màn hình có thể khiến người xem nghĩ nút đó quan trọng hơn.
- **Câu hỏi lựa chọn**
  - Vị trí nút có ý nghĩa dữ liệu hay chỉ là kết quả của thuật toán?
  - Khoảng cách giữa hai nút có biểu diễn khoảng cách thực không?
  - Nút lớn có thực sự quan trọng hay chỉ có một giá trị cao?
  - Việc thay đổi bố cục có làm thay đổi kết luận không?
  - Các nút hoặc liên kết nào đã bị lọc khỏi biểu đồ?

Hướng dẫn lựa chọn biểu đồ trực quan

PHENIKAA UNIVERSITY
School of Computing

Câu hỏi phân tích	Dạng trực quan đề xuất
Cấu trúc gồm những cấp nào?	Tree diagram hoặc sunburst
Nhóm nào chiếm tỷ trọng lớn?	Treemap
Danh mục nào có giá trị lớn hơn?	Biểu đồ thanh
Các đối tượng kết nối với nhau thế nào?	Node-link diagram
Mạng dày đặc có những cụm nào?	Adjacency matrix
Luồng đi từ đâu đến đâu?	Sankey diagram
Nhóm thay đổi qua nhiều giai đoạn thế nào?	Alluvial diagram
Quá trình giảm dần qua các bước ra sao?	Funnel chart hoặc Sankey
Nút nào có nhiều liên kết nhất?	Biểu đồ thanh theo bậc
Giá trị chính xác của từng luồng là bao nhiêu?	Bảng hoặc biểu đồ thanh bổ trợ

5. Thực hành

Trắc nghiệm

PHENIKAA UNIVERSITY
School of Computing

Câu 1: Biểu đồ nào phù hợp nhất để thể hiện cấu trúc thư mục?

A. Sankey diagram
B. Tree diagram
C. Scatter plot
D. Histogram

Câu 2: Trong treemap, giá trị định lượng thường được biểu diễn bằng:

A. Diện tích
B. Độ dốc
C. Vị trí trên trục thời gian
D. Hình dạng đường

Trắc nghiệm

PHENIKAA UNIVERSITY
School of Computing

Câu 3: Trong mạng có hướng, số cạnh đi vào một nút được gọi là:

A. Out-degree
B. In-degree
C. Weight
D. Depth

Câu 4: Biểu diễn nào không có hiện tượng các cạnh chồng lên nhau?

- A. Node-link diagram
  B. Adjacency matrix
  C. Sankey diagram
  D. Tree diagram

Trắc nghiệm

PHENIKAA UNIVERSITY
School of Computing

Câu 5: Độ rộng của dải trong Sankey thường biểu diễn:

A. Thời gian
B. Khoảng cách
C. Khối lượng luồng
D. Số cấp phân cấp

Câu 6: Khi cần so sánh chính xác doanh thu của các danh mục, biểu đồ nào thường phù hợp hơn treemap?

A. Biểu đồ thanh
B. Sunburst
C. Node-link diagram
D. Sankey diagram

Trắc nghiệm

PHENIKAA UNIVERSITY
School of Computing

Câu 7: Hạn chế lớn của node-link diagram khi dữ liệu lớn là gì?

A. Không biểu diễn được nút
B. Không sử dụng được màu sắc
C. Nút và cạnh dễ chồng lấn
D. Không biểu diễn được mạng có hướng

Câu 8: Alluvial diagram thường được sử dụng khi cần:

A. So sánh trung bình của hai nhóm
B. Thể hiện sự thay đổi nhóm qua nhiều giai đoạn
C. Biểu diễn phân phối một biến
D. Thể hiện tương quan giữa hai biến số



PHENIKAA UNIVERSITY
School of Computing

Thank You!

Q&A